

Roteamento/Endpoints

Objetivo da Aula

Compreender os conceitos relacionados a roteamento e endpoints, além de aplicar o conhecimento adquirido na construção de Web Services RESTful.

Apresentação

Em sistemas antigos, a navegação era baseada apenas em links entre as páginas e ações de formulários, mas isso se mostrou inadequado para garantir a segurança dos aplicativos. Passamos a restringir os endereços capazes de acessar o servidor, juntamente com o método HTTP adotado, o que passou a ser conhecido como rota.

Veremos, assim, o conceito de endpoint – um ponto de acesso para fornecimento de serviços, que permite o tratamento de rotas permitidas no servidor, incluindo a forma parametrizada – e como o Express permite organizar o roteamento de um sistema para web, associando endpoints e métodos HTTP.

Ao final, utilizaremos os conhecimentos adquiridos para a construção de um Web Service RESTful simples, mas que trata todos os métodos disponíveis no protocolo HTTP, oferecendo serviços de consulta, inclusão, alteração e exclusão, a partir de um repositório em memória.

1. Conceito de Rotas e Endpoints

A arquitetura REST utiliza o modelo de comunicação do protocolo HTTP, com base em endereços e métodos de acesso. Dessa forma, os serviços se integram naturalmente com o ambiente, além de utilizarem JSON para a transferência de dados, garantindo a interoperabilidade.

Quando falamos de **endpoint**, estamos nos referindo ao caminho que leva a um recurso de forma genérica. Suas APIs deverão fornecer os endpoints para que outros sistemas utilizem os serviços disponibilizados – por exemplo, um serviço de informações meteorológicas a partir do **CEP**, em que um endpoint compatível poderia ser **consultacep/:cep**.

Note que os endpoints definem acessos a recursos genéricos, e nossos sistemas devem ser capazes de capturar diferentes requisições nesses endpoints, através de diferentes métodos do HTTP. Nesse contexto, temos a **rota**, que trata da utilização do endpoint através de uma URL. Considerando o exemplo do CEP, uma rota seria **GET http://servidor/consultacep/20330200**.

Na prática, através do Express, definimos um endpoint e o método HTTP que é aceito, e qualquer rota compatível será tratada pela callback. É comum nos referirmos a esses endpoints como rotas, mas devemos ter em mente que os endpoints são elementos mais abstratos, utilizados na construção das rotas.

Considerando os endpoints **/notas** e **/notas/:id**, utilizados em um serviço local, segundo a arquitetura REST, teríamos rotas como as do quadro seguinte.

Quadro 1: Exemplos de endpoints e rotas relacionadas na arquitetura REST

Endpoint	Rota	Descrição
/notas	GET http://localhost:3000/notas	Consulta todas as notas.
/notas	POST http://localhost:3000/notas	Inclui uma nota, com os dados enviados no corpo da requisição.
/notas/:id	GET http://localhost:3000/notas/1	Consulta a nota com id 1.
/notas/:id	GET http://localhost:3000/notas/2	Consulta a nota com id 2.
/notas/:id	PUT http://localhost:3000/notas/1	Altera a nota com id 1, utilizando os dados do corpo da requisição.
/notas/:id	DELETE http://localhost:3000/notas/2	Remove a nota com id 2.

Fonte: Elaboração própria.

2. Geração das Entidades

Para implementar um Web Service **RESTful**, precisamos de persistência em banco de dados, e aqui vamos utilizar o **MySQL** como banco de dados. Ele deverá estar instalado em sua máquina para que os códigos apresentados funcionem.

Nosso serviço irá lidar com uma entidade simples, denominada **Nota**, que terá como atributos o **título** e a **descrição**. Embora sejam poucos campos, as técnicas apresentadas serão válidas para qualquer entidade mais complexa.

Crie um diretório vazio, abra no VS Code e execute os comandos da listagem seguinte, através do terminal interno.

```
npm init -y
npm install sequelize mysql2
npm install --save-dev sequelize-cli
npx sequelize-cli init
```

Os comandos estão iniciando o projeto, incluindo os módulos do **Sequelize** e do **MySQL**, além de instalar o aplicativo de linha de comando do Sequelize no projeto. Na última linha, esse aplicativo é utilizado para gerar a estrutura inicial de persistência no projeto.

O próximo passo é configurar o acesso ao banco, no arquivo **config.json**, para o ambiente de desenvolvimento (development), como no exemplo seguinte.

```
"development": {
  "username": "root", "password": "admin123",
  "database": "agenda_dev", "host": "127.0.0.1",
  "dialect": "mysql"
},
```

Lembre-se de alterar a **senha** do root para a sua senha, e aqui estamos considerando o MySQL de forma local, mas poderia ser utilizado outro valor para **host**, indicando um endereço externo. Faça as mesmas alterações nas configurações de teste (test) e produção (production), apenas mudando o nome do banco (**database**).

Agora, você deve criar o banco de dados, executando o comando a seguir.

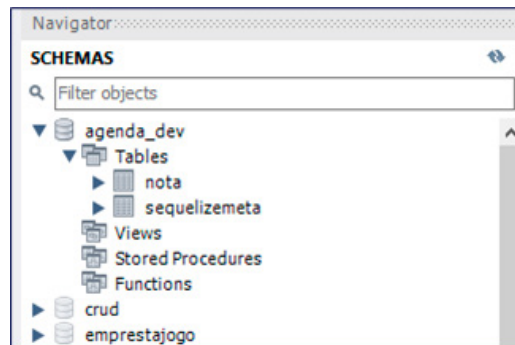
```
npx sequelize-cli db:create
```

Com a execução do comando, o banco de dados será criado no MySQL, e já podemos definir nossas entidades, utilizando os comandos da listagem seguinte.

```
npx sequelize-cli model:generate --name Nota
--attributes titulo:string,descricao:string
npx sequelize-cli db:migrate
```

O primeiro comando cria a entidade **Nota**, no diretório **models**, de acordo com a arquitetura de persistência definida pelo **Sequelize-CLI**. Também são gerados os arquivos de migração, ou **migrations**, para criação das estruturas no banco de dados, e, no segundo comando, que utiliza **db:migrate**, essas migrações são acionadas, gerando uma tabela no MySQL.

Figura 1: Tabela criada no MySQL



Fonte: Elaboração própria.

3. Roteamento no Express

Nossas entidades já estão prontas, mas precisamos compreender um pouco mais acerca do sistema de roteamento do Express antes de prosseguir.

O servidor Express oferece múltiplas formas de interpretação de rotas pelos endpoints, permitindo configurações muito robustas e flexíveis. Podemos definir endpoints por **roteamento direto**, nos métodos de tratamento do objeto Express, como nos exemplos da listagem seguinte.

```
app.get('/about', (req, res) => {res.send('método GET');});
app.put('/ab?cd/:id', (req, res) => {res.send('método PUT');});
app.post('/ab*cd', (req, res) => {res.send('método POST');});
app.delete('/ab?d', (req, res) => {res.send('método DELETE');});
```

Para todos os exemplos, temos o mesmo retorno simples, indicando o método HTTP que foi utilizado na requisição. Por outro lado, os endpoints oferecem uma rica diversidade de opções para as rotas.

No quadro seguinte, temos alguns endpoints e as rotas aceitas, excluindo o método HTTP e o endereço de base do servidor.

Quadro 2: Exemplos de endpoints e rotas possíveis no Express

Endpoint	Rotas relativas
/about	/about
/ab?cd	/acd, /abcd
/ab+cd	/abcd, /abbc, /abbbcd,...
/ab*cd	/abcd, /ab123cd, /abRANDOMcd,...
/ab(cd)?e	/abe, /abcde
/. *fly\$/	/butterfly, /dragonfly,...

Fonte: Elaboração própria.

Outra opção interessante do Express é o uso de **middleware** na forma de filtros, segundo o padrão Filter Chain, em que um endpoint pode responder com uma sequência de callbacks, em vez de apenas um tratamento simples. Essa abordagem permite, por exemplo, a verificação de direitos de acesso antes de fornecer a informação para o usuário.

```
const cb0 = (req, res, next) => {
  console.log('CB0'); next();
}

const cb1 = (req, res, next) => {
  console.log('CB1'); next();
}

const cb2 = (req, res) => {res.send('Hello from C!');}

app.post('/example/c', [cb0, cb1, cb2]);
```

Observando a listagem de exemplo, temos três callbacks, que são utilizadas na resposta a uma chamada para **http://localhost/example/c**, como pode ser observado no vetor, logo após o endpoint. As duas primeiras callbacks incluem o parâmetro **next**, permitindo que o método de mesmo nome direcione o fluxo para a próxima callback do vetor.

Caso o tratamento fosse utilizado em um servidor Express, ao receber uma requisição, seria acionada a callback **cb0**, que imprime **CB0** no console e, depois, repassa o fluxo para

cb1, imprimindo **CB1** e, finalmente, direcionando para **cb2**, em que é retornada a resposta via método **send**.

Você pode configurar a resposta para vários métodos HTTP simultâneos, com a utilização do **método route**.

```
app.route('/book')
  .get(function(req, res) { res.send('Get a random book'); })
  .post(function(req, res) { res.send('Add a book'); })
  .put(function(req, res) { res.send('Update the book'); });
```

No exemplo, temos respostas diferenciadas para os métodos GET, POST e PUT do HTTP, através dos métodos com nomes equivalentes, mas o **endpoint** é o mesmo para os três acessos. Em uma arquitetura REST, seria uma forma prática para tratar a consulta completa e a inclusão simultaneamente.

Você também pode aceitar todos os métodos do HTTP via **método all**.

```
app.all('/secret', function (req, res, next) {
  console.log('Accessing the secret section...'); next();
});
```

Certamente, a forma mais organizada de trabalhar no Express é através de um **objeto Router**. Ele concentra endpoints relativos, que podem ser acrescentados de um prefixo ao nível do objeto Express.

```
let router = express.Router();
router.use(function timeLog(req, res, next) {
  console.log('Time: ', Date.now()); next();
});
router.get('/', function(req, res) {
  res.send('Birds home page');
});
app.use('/birds', router);
```

O objeto Router é instanciado no início do código, utiliza um middleware que escreve a data e hora de acesso no console para todas as requisições e define o tratamento para a rota **raiz** via método **GET**. Através do método **use**, o objeto de roteamento é utilizado pelo Express, sendo definido o prefixo **/birds**.

Com o servidor ativo, um acesso ao endereço **http://localhost:3000/birds** iria imprimir a mensagem no console, devido ao uso do middleware, e retornar para o usuário com a resposta prevista no roteador.

4. Construção de Web Service RESTful

Em uma arquitetura REST, os serviços relacionados às rotas irão gerenciar as entidades e, por consequência, os registros no banco de dados. Como nossa camada de persistência já está pronta, vamos executar os últimos passos na construção de nosso Web Service, no qual utilizaremos uma biblioteca externa ao Express para a codificação do JSON.

Adicione as bibliotecas necessárias, através dos comandos seguintes.

```
npm install express
npm install body-parser
```

Agora, precisamos definir a estrutura básica do servidor, no arquivo **index.js**, que deve ser criado na raiz do projeto, utilizando o conteúdo seguinte.

```
const express = require('express');
const bodyParser = require('body-parser');
const app = express();
app.use(bodyParser.json());
app.listen(3000, () => console.log("Servidor pronto"));
```

Podemos observar uma implementação bastante simples, em que instanciamos um objeto do tipo **Express**, associamos o codificador **JSON** de **Body Parser** e iniciamos a execução com o método **listen**, utilizando a porta **3000**. Como ainda não configuramos um objeto de roteamento, ao executar o servidor, via comando **node index**, teremos apenas a mensagem indicando servidor pronto no console, e qualquer acesso retornará à página padrão de erro.

Vamos iniciar a codificação do arquivo **rotas.js**, com o conteúdo apresentado na listagem seguinte.

```
const express = require('express');
const db = require('./models/index.js');
const router = express.Router();
router.get('/', async(req,res)=>{
  let dados = await db.Nota.findAll();
  res.json(dados);
})
router.get('/:id', async(req,res)=>{
  let dados = await db.Nota.findByPk(req.params.id);
  res.json(dados);
})
```

Após instanciar o objeto **router**, definimos os endpoints para **consulta**, via método **GET** do HTTP. Note que usamos o modelo assíncrono, através de **async** e **await**, e que temos uma rota de base e uma rota parametrizada com o **id**.

Para a rota de base, é efetuada uma consulta ao banco, via método **findAll**, retornando todas as notas da base de dados no formato JSON. De forma similar, a rota parametrizada captura o **id** fornecido, através de **req.params**, e retorna a nota específica no formato JSON, com a consulta efetuada via **findByPk**.

Agora, vamos acrescentar o tratamento do método POST, na rota de base, adicionando o fragmento de código seguinte no arquivo **rota.js**.

```
router.post('/', async(req,res)=>{
  let dados = await db.Nota.create(req.body);
  res.json(dados);
})
```


Os dados são enviados no formato JSON, sendo capturados via **req.body**, e o registro é gerado na base de dados através do método **create**. Como retorno, temos os dados da nota criada, incluindo o **id** que foi gerado automaticamente.

Para os métodos PUT e DELETE, utilizamos a rota parametrizada. Acrescente o trecho de código seguinte no arquivo **rota.js**.

```
router.delete('/:id', async(req,res)=>{
  let dados = await db.Nota.destroy(
    {where: {id: req.params.id}});
  res.end();
})
router.put('/:id', async(req,res)=>{
  let dados = await db.Nota.update(
    req.body,
    {where: {id: req.params.id}});
  res.end();
})
module.exports = router
```

Em ambos os métodos, é feita uma restrição de consulta, recuperando apenas o registro com **id** equivalente ao fornecido pela **rota**. Para o método DELETE, temos a remoção do registro, via método **destroy**, e, para o PUT, substituímos os valores do registro por aqueles fornecidos no corpo da requisição, utilizando o método **update** e o atributo **req.body**.

O retorno, em ambos os casos, será vazio, mas o resultado pode ser testado através do código de resposta, que será **200** para operações bem-sucedidas.

Ao final do arquivo, o roteador é **exportado**, e devemos utilizá-lo em **index.js**, alterando o código para o que é apresentado a seguir.

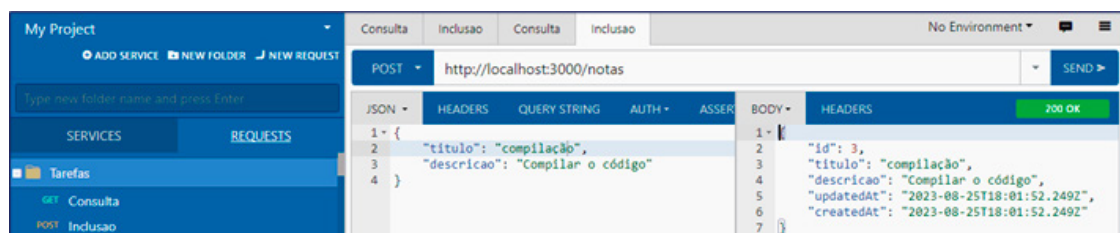
```
const express = require('express');
const bodyParser = require('body-parser');
const rotaNotas = require('./rotas')
```

```
const app = express();
app.use(bodyParser.json());
app.use('/notas', rotaNotas);
app.listen(3000, () => console.log("Servidor pronto"));
```

As únicas modificações foram a importação do roteador, utilizando o nome **rotaNotas**, e o uso dele pelo objeto Express a partir do endpoint **/notas**. Com isso, os endereços serão iniciados por **http://localhost:3000/notas**.

Você poderá executar o servidor com o comando **node index** e testar todas as operações por alguma ferramenta como o plugin **Boomerang**.

Figura 2: Inclusão de nota através do Boomerang



Fonte: Elaboração própria.

Para a consulta, pode ser usado um navegador comum, já que utiliza o método GET do HTTP.

Figura 3: Teste da consulta pelo navegador



Fonte: Elaboração própria.

Considerações Finais da Aula

Atualmente, temos uma grande heterogeneidade de tecnologias no ambiente web, e a interoperabilidade é um fator de sucesso ao implementar o back-end do sistema. Nesse contexto, o uso de APIs, no estilo REST, permite a garantia de interoperabilidade, ao trabalhar com o formato de texto JSON na comunicação e adotar o protocolo HTTP para acesso.

Conhecemos o conceito de endpoint e vimos como são utilizados, junto aos métodos do protocolo HTTP, para a definição das rotas disponibilizadas pelo servidor. Como o REST é baseado em rotas para acesso aos recursos e utiliza os métodos do HTTP para especificar o tipo de operação que será realizado, foi necessário compreender o roteamento do Express de forma mais profunda, com suporte aos métodos GET, POST, PUT e DELETE, além de parametrizações.

Embora ainda não tenhamos a persistência em banco de dados, foi possível definir um Web Service RESTful simples, dando suporte a todas às operações de consulta, inclusão, alteração e exclusão para um tipo de entidade, apenas com a restrição de um repositório em memória. Mesmo utilizando um exemplo simples, foi possível testar todas as funcionalidades de uma API no estilo REST.

Materiais Complementares

Artigo

O que são endpoints e rotas de uma api

2023, Igor Alves.

O artigo descreve os conceitos de endpoints e de rotas para uma API REST, com diversos exemplos para facilitar o entendimento.

Link para acesso: <https://www.dio.me/articles/o-que-sao-endpoints-e-rotas-de-uma-api> (acesso em: 13 set. 2023).

Artigo

Criando uma API RESTful com NodeJS e Express

2019, Tiago Lima.

Artigo descrevendo a criação de um Web Service RESTful, no ambiente do NodeJS, com utilização do framework express.

Link para acesso: <https://medium.com/xp-inc/https-medium-com-tiago-jlima-developer-criando-uma-api-restful-com-nodejs-e-express-9cc1a2c9d4d8> (acesso em: 13 set. 2023).

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKdl>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKdK>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKdO>. Acesso em: 12 set. 2023.